Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Classes and Objects

Introduction to Computer Programming

Management and Artificial Intelligence

LUISS



Describing a Car

We can use a dictionary to keep a car's data, and ordinary functions to work with it:

```
car = {"make": "Toyota", "color": "blue", "speed": 0}

def accelerate(car):
    car["speed"] += 10

def brake(car):
    car["speed"] = max(0, car["speed"] - 10)

accelerate(car)
print(car["speed"])  # 10
```

The car's data and the functions that operate on it are separate.

Describing a Persona

We can describe a person in the same way, keeping the data in a dictionary and using a function to describe them:

```
person = {"first_name": "Ada", "last_name": "Lovelace", "age": 36}

def describe(person):
    return f"{person['first_name']} {person['last_name']}, age {person['age']}"

print(describe(person))
```

This works, but we have to keep track of which functions go with which data. Classes give us another way to organize them.

Objects and Classes

Recall from our first Python lecture that a given data type is characterised by:

- a set of **values** (the domain)
- a set of **operations** that allow to manipulate domain elements
- some **constants** (special domain values)

For example, in Python we have seen:

- **integers:** where the *domain* is the set of integer numbers and *operations* are arithmetic operations (e.g., `+`, `-`, `*`, `**`, `/`, `//`)
- **floating point:** where the *domain* is the set of real numbers and *operations* are arithmetic operations (e.g., `+`, `-`, `*`, `**`, `/`, `//`)
- **strings:** where the *domain* is the set of sequences of characters and *operations* are string concatenation, replication, substring extraction, and so on.

Python is an **object-oriented programming language**: the programmer can define its own data types, that are known as **classes**.

Python is an **object-oriented programming language**: the programmer can define its own data types, that are known as **classes**.

If C is a class, a value of a class C is called an *object*. In other words, an object is an *instance* of the class C and C can be seen as a blueprint for that object.

Python is an **object-oriented programming language**: the programmer can define its own data types, that are known as **classes**.

If C is a class, a value of a class C is called an *object*. In other words, an object is an *instance* of the class C and C can be seen as a blueprint for that object.

Each object contains:

- **instance variables**: i.e., variables that are used to represent a domain value;
- **methods**: i.e., functions through which domain elements can be manipulated.

Python is an **object-oriented programming language**: the programmer can define its own data types, that are known as **classes**.

If C is a class, a value of a class C is called an *object*. In other words, an object is an *instance* of the class C and C can be seen as a blueprint for that object.

Each object contains:

- **instance variables**: i.e., variables that are used to represent a domain value;
- **methods**: i.e., functions through which domain elements can be manipulated.

For example, in Python we have seen:

- **lists:** a bunch of items with methods like `sort()`, `reverse()`, `extend()`
- **dicts:** a bunch of key-value pairs with methods like `update()`, `pop()`, `clear()`
- **strings:** a bunch of characters with methods like `split()`, `index()`, `find()`



Where possible methods may include:

- steer left
- steer right
- brake
- ...

Defining a Class

Like function definitions begin with the `def` keyword in Python, class definitions begin with the `class` keyword.

```
class Person:
    """ Docstring of the class """

    # place here all variables
    age = 10

    # place here all methods
    def greet(self):
        print("Hello")
```

As soon as we define a class, a new *class object* is created with the same name. This special *class object* allows us to see the attributes of the class (variables and/or methods) and, of course, to create new objects for that class.

We can therefore now create new objects for that class:

```
charlie = Person() # instantiation of a new object
print(type(charlie))
```

```
<class '__main__.Person'>
```

We can therefore now create new objects for that class:

```
charlie = Person() # instantiation of a new object
print(type(charlie))
```

```
<class '__main__.Person'>
```

And we can also access to its attributes (methods and/or variable) via the well-known dot notation:

```
print(charlie.age)
charlie.greet()
```

```
10
Hello
```

We can therefore now create new objects for that class:

```
charlie = Person() # instantiation of a new object
print(type(charlie))
```

```
<class '__main__.Person'>
```

And we can also access to its attributes (methods and/or variable) via the well-known dot notation:

```
print(charlie.age)
charlie.greet()
```

```
10
Hello
```

NOTE: Instantiating a new object is pretty much similar to calling a function.

self

You may have noticed the `self` parameter in the function definition inside the class.

```
class Person:
    ...
    def greet(self):
        print("Hello")
```

Yet, we called the method simply as `charlie.greet()` without any arguments.

This is because, whenever an object calls its method, the object itself is passed as the first argument. Hence:

```
charlie.greet()
```

translates into:

```
Person.greet(charlie)
```

The first argument of a class method must be the object itself. This is conventionally called `self`.

Constructor

Initialising an internal variable as we did:

```
charlie.name = "Charlie"
```

is not very flexible.

We can use a special function called `__init__()` which is called whenever a new object is instantiated.

Commonly, it is used to initialise the internal variables of an object. In Object-Oriented-Programming, this is called a **constructor**.

```
class Person:
    def __init__(self, firstName, lastName, age, address=None):
        self.first_name = firstName
        self.last_name = lastName
        self.address = address
        self.age = age
```

Complete Example

```
class Person:
    def __init__(self, firstName, lastName, age=None, address=None):
        self.first_name = firstName
        self.last_name = lastName
        self.address = address
        self.age = age
    def greet(self):
        print("Hello!")
        print("My name is", self.first_name, self.last_name)
        print("I am", self.age, "years old")
        print("I live in", self.address)
```

```python
person1 = Person("Francesco", "Totti", 30, "Viale Romania")
person2 = Person("Daniele", "De Rossi")

person1.greet()
person2.greet()
```

```
Hello!
My name is Francesco Totti
I am 30 years old
I live in Viale Romania
Hello!
My name is Daniele De Rossi
I am None years old
I live in None
```

Note: You can pass input parameters to methods as well.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def description(self):
        return self.name + " is " + str(self.age) + " years old"
    def speak(self, sound):
        return self.name + " says " + sound

miles = Dog("Miles", 4)
print(miles.description())
print(miles.speak("Woof Woof"))
print(miles.speak("Bow Wow"))
```

```
Miles is 4 years old
Miles says Woof Woof
Miles says Bow Wow
```

Modifying Objects

So far we have seen:

- *static* classes, e.g. `Person`: we initialise it with some values and that's it;
- *less static* classes, e.g. `Dog`: we initialise it with some values, but we can also change the behaviour of methods at run-time.

Now, the big question is: *can we change the inner variables of an object?*

Short answer? **Yes!**

Example

Let's define a data type to count events. Think for example of a physical object, with a button: every time you press the button, the counter increases by 1. How can we model it?

Example

Let's define a data type to count events. Think for example of a physical object, with a button: every time you press the button, the counter increases by 1. How can we model it?

- The *domain* of our data type is the set of natural numbers
- The *constant* is 0 (i.e., a counter that has not counted anything yet)
- The *operations* are:
    - `__init__()`, which initialises the counter to 0 (i.e., a brand new counter)
    - `increase()`, which increases the counter by 1
    - `get_count()`, which reads the counter (i.e., returns the current count)

Example

Let's define a data type to count events. Think for example of a physical object, with a button: every time you press the button, the counter increases by 1. How can we model it?

- The *domain* of our data type is the set of natural numbers
- The *constant* is 0 (i.e., a counter that has not counted anything yet)
- The *operations* are:
    - `__init__()`, which initialises the counter to 0 (i.e., a brand new counter)
    - `increase()`, which increases the counter by 1
    - `get_count()`, which reads the counter (i.e., returns the current count)

In order to represent the domain value (i.e., the counter internal state) we define a variable called `_count`.

NOTE: The leading underscore conventionally states that it is an "internal", private variable of each instance.

Possible solution:

```
class Counter:
    def __init__(self):
        self._count = 0
    def increase(self):
        self._count += 1
    def get_count(self):
        return self._count
```

Possible solution:

```
class Counter:
    def __init__(self):
        self._count = 0
    def increase(self):
        self._count += 1
    def get_count(self):
        return self._count
```

RECALL: Methods are defined exactly like ordinary functions, but inside the `class` block: - they can have any number of parameters - the first parameter, that by convention is called `self`, has a special meaning as it contains a reference to the instance.

Possible solution:

```
class Counter:
    def __init__(self):
        self._count = 0
    def increase(self):
        self._count += 1
    def get_count(self):
        return self._count
```

RECALL: Methods are defined exactly like ordinary functions, but inside the `class` block: - they can have any number of parameters - the first parameter, that by convention is called `self`, has a special meaning as it contains a reference to the instance.

RECALL: As `self` refers to the instance, we can access its variables via `self.`.

And here's how to use it:

```
c = Counter()                              # Line 1
c.increase()                               # Line 2
c.increase()                               # Line 3
state = c.get_count()                      # Line 4
print("I have counted", state, "times") # Line 5
```

```
I have counted 2 times
```

What’s going on here?

And here's how to use it:

```
c = Counter()                              # Line 1
c.increase()                               # Line 2
c.increase()                               # Line 3
state = c.get_count()                      # Line 4
print("I have counted", state, "times") # Line 5
```

```
I have counted 2 times
```

What’s going on here?

1. on line 1, we create a new `Counter` object. Instantiation automatically calls the `__init__()` method, which initialises the `_count` variable to `0`.

And here's how to use it:

```
c = Counter()                              # Line 1
c.increase()                               # Line 2
c.increase()                               # Line 3
state = c.get_count()                      # Line 4
print("I have counted", state, "times") # Line 5
```

```
I have counted 2 times
```

What’s going on here?

1. on line 1, we create a new `Counter` object. Instantiation automatically calls the `__init__()` method, which initialises the `_count` variable to `0`.

2. on line 2, we trigger the `increase` method, which increments `c._counter` by 1.

And here's how to use it:

```
c = Counter()                              # Line 1
c.increase()                               # Line 2
c.increase()                               # Line 3
state = c.get_count()                      # Line 4
print("I have counted", state, "times") # Line 5
```

```
I have counted 2 times
```

What's going on here?

1. on line 1, we create a new `Counter` object. Instantiation automatically calls the `__init__()` method, which initialises the `_count` variable to `0`.

2. on line 2, we trigger the `increase` method, which increments `c._counter` by 1.

3. on line 3, we trigger the `increase` method, which increments `c._counter` by 1.

And here's how to use it:

```
c = Counter()                              # Line 1
c.increase()                               # Line 2
c.increase()                               # Line 3
state = c.get_count()                      # Line 4
print("I have counted", state, "times") # Line 5
```

```
I have counted 2 times
```

What's going on here?

1. on line 1, we create a new `Counter` object. Instantiation automatically calls the `__init__()` method, which initialises the `_count` variable to `0`.

2. on line 2, we trigger the `increase` method, which increments `c._counter` by 1.

3. on line 3, we trigger the `increase` method, which increments `c._counter` by 1.

4. on lines 4 and 5, we trigger the `get_count` method, which returns the current state of the counter (i.e., `c._counter`) and then we print it.

Static vs Instance Methods

Some functions **belong conceptually to a class**, but **don't need access** to instance data.

Python supports different types of methods within a class:

- **Instance methods**: These methods take `self` as the first parameter and can access/modify instance data. They are the most common type of methods in classes.
- **Static methods**: These methods don't take `self` as a parameter and cannot access instance attributes. They are defined using the `@staticmethod` decorator and behave like regular functions that belong to the class namespace.

Method Type	Needs `self`?	Access instance data?	Typical Use
Instance Method	Yes	Yes	Operate on object data
Static Method	No	No	Utility/helper functions, Alternate constructors

```
class Value:
    def __init__(self, value):
        self._value = value

    def value(self):
        return self._value

    def add(self, x):
        return self._value + x

    def multiply(self, x):
        return self._value * x

    @staticmethod
    def max(values):
        if not values:
            return None
        max_value = values[0]
        for v in values[1:]:
            if v.value() > max_value.value():
                max_value = v
        return max_value
```

```python
# Using instance methods
v = Value(5)
print(v.add(10))      # Output: 15
print(v.multiply(3))  # Output: 15

# Using static methods
v2 = Value(50)
v3 = Value(20)
max_value = Value.max([v, v2, v3])
print(max_value.value())
```

```
15
15
50
```

Encapsulation

NOTE:

We may as well increase `c._counter` by hand, like so: `c._counter += 1` instead of calling `c.increase()`.

Can we do it? **Yes**. Should we do it? **No**.

Why? Because we seek to separate:

- a public interface of the class, well described to the end-users via its methods by which we can manipulate objects;
- the inner working of the class, which is private to the class designer and should not be seen (or changed) from "outside of the box".

This principle is known as ***encapsulation***.

Encapsulation works in the physical world too!

EXAMPLE: You can use a car by manipulating its public interfaces (pedals, steering wheel, handbrake, ...) -- which are mostly standardised across models.

You do not have (and you probably do not want to) access internal parts of your car to operate it!

Python, unlike other programming languages, does not forbid the end-user to access private attributes (which by convention begin with a _).

If you wish to make it more difficult (but not impossible) to access to a private attribute, use double-underscores __.

Inheritance

Inheritance refers to defining a new class with little or no modification to an existing class:

- the new class is called *derived class* (or *child class*);
- the one from which it inherits is called the *base class* (or *parent class*).

Example: we want to create a class `Student` to represent university students. We want to represent data such as the student ID, the university name and the average grade using instance variables.

Example: we want to create a class `Student` to represent university students. We want to represent data such as the student ID, the university name and the average grade using instance variables.

NOTE:

But a student has also a last name, a first name, etc., like any person; because a student is a person! Name, surname, age and address are already available in the `Person` class.

Can we reinvent the wheel? **Yes**. Should we do it? **Nah**.

```python
class Person:
    def __init__(self, firstName, lastName, age=None, address=None):
        self.first_name = firstName
        self.last_name = lastName
        self.address = address
        self.age = age
    def greet(self):
        print("Hello!")
        print("My name is", self.first_name, self.last_name)
        print("I am", self.age, "years old")
        print("I live in", self.address)
```

We would like to define class `Student`, so that the following property holds:

Every instance of class `Student` is also an instance of class `Person`.

In other words, we have detected an **is-a** relation between students and persons (because a student is-a person), and we would like to model this relation in our Python program.

In Python is-a relations come to life thanks to inheritance. Let's define a class `Student` that extends class `Person`:

```
class Student(Person):
    pass # pass is a Python keyword that means "do nothing"

student1 = Student("Alessio", "Martino", 30, "Viale Romania")
print(student1.last_name)
student1.greet()
print(type(student1))
```

```
Martino
Hello!
My name is Alessio Martino
I am 30 years old
I live in Viale Romania
<class '__main__.Student'>
```

`Person` is known as parent class of `Student`. As such, every instance of class `Student` inherits methods and instance variables from its parent class.

NOTE: You can inherit from multiple classes, e.g.:

`UniversityTeamPlayer(Student, Athlete)`

Every instance of `UniversityTeamPlayer` inherits from `Student` and `Athlete`. This is a slightly more advanced topic, yet it's good to know.

HINT: You can surely use the `type()` function to check the class of an object. A clever alternative, which accounts for inheritance, is the `isinstance()` function.

The `isinstance(obj, c)` function is a predicate that tests whether an object `obj` is an instance of class `c`.

```
Mary = Person('Stuart', 'Mary', 50)
Mark = Student('Holleymore', 'Mark', 22)
print(isinstance(Mary, Person))
print(isinstance(Mary, Student))
print(isinstance(Mark, Person))
print(isinstance(Mark, Student))
```

```
True
False
True
True
```

What's going on here?

- we defined Mary as a `Person`, so she is not a `Student`
- we defined Mark as a `Student`, but a student is a person, so both of his `isinstance()` calls yield `True`

PROBLEM: There is still no difference between `Student` and `Person`.

PROBLEM: There is still no difference between `Student` and `Person`.

Indeed, we still have to enrich the class `Student` in order to accomodate student ID, university name and grades.

```
class Student(Person):
    def __init__(self,name,surname,uni,studID,age=None,addr=None):
        Person.__init__(self,name,surname,age,addr)
        self.university = uni
        self.studentID = studID
        self.exam_grades = []

    def add_exam(self,grade):
        self.exam_grades.append(grade)

    def get_average(self):
        return sum(self.exam_grades) / len(self.exam_grades)
```

What's going on here?

- `Student` has a new `__init__()` that overwrites the one in `Person`
- when initialising `Student`, some input arguments that are proper to `Person` are fed to `Person.__init__()` so we can re-use its `__init__()`
- `Student` is initialised with two additional variables in order to account for the university name (`uni`) and the student ID (`studID`)
- `Student` also features a third variable (`exam_grades`) that collects the grades in a list
- `Student` also features two dedicated methods:
    - `add_exam()` to add grades to the student's curriculum
    - `get_average()` to calculate the average student grade

WARNING:

The is-a relation is a one-way relation: in our case, all students are persons, but not all persons are students.

Practically speaking, `Student` inherits from `Person`, but `Person` does not inherit from `Student`.

Let's see our brand new `Student` class in action.

```
student1 = Student("Francesco", "Totti", "LUISS", "123456", 30, "Viale Romania")

# what we inherited from Person
print(student1.first_name)
student1.greet()

# what we defined in Student
student1.add_exam(30)
student1.add_exam(25)
student1.add_exam(18)
print(student1.get_average())
```

```
Francesco
Hello!
My name is Francesco Totti
I am 30 years old
I live in Viale Romania
24.333333333333332
```

Using `super()` in child classes

In the `Student` constructor we called the parent constructor explicitly:

```
class Student(Person):
    def __init__(self, name, surname, uni, studID, age=None, addr=None):
        Person.__init__(self, name, surname, age, addr)
        self.university = uni
        self.studentID = studID
        self.exam_grades = []
```

A more idiomatic way in Python is to use the built-in `super()` function:

```
class Student(Person):
    def __init__(self, name, surname, uni, studID, age=None, addr=None):
        super().__init__(name, surname, age, addr)
        self.university = uni
        self.studentID = studID
        self.exam_grades = []
```

- `super()` returns an object that represents the parent class *for this specific class*.
- Calling `super().__init__(...)` automatically finds and calls the right parent constructor (`Person.__init__` here).

Why is `super()` preferred over `Person.__init__(self, ...)`?

- **Less repetition:** you don't have to hard-code the parent class name.
- **Easier to change hierarchy:** if you later rename `Person` or change the parent class, `super()` keeps working.
- **Multiple inheritance friendly:** with more than one parent, `super()` follows Python's method resolution order (MRO) and cooperates with other parent classes.

Polymorphism

NOTE: In the child class you can overwrite any methods, not just `__init__()`.

Polymorphism

NOTE: In the child class you can overwrite any methods, not just `__init__()`.

This aspect introduces us to the concept of **polymorphism**.

Polymorphism

NOTE: In the child class you can overwrite any methods, not just `__init__()`.

This aspect introduces us to the concept of **polymorphism**.

Broadly speaking, in computer programming, polymorphism means the same function name (but different signatures) being used for different types.

Polymorphism

NOTE: In the child class you can overwrite any methods, not just `__init__()`.

This aspect introduces us to the concept of **polymorphism**.

Broadly speaking, in computer programming, polymorphism means the same function name (but different signatures) being used for different types.

In Python, common polymorphic functions include:

- `len()`, that returns the number of characters in a string, the number of items in a tuple/list, the number of key-value pairs in a dictionary
- `sum()`, which adds the elements of a sequence as long as the elements of the sequence support addition.

Let's suppose that our `greet()` does not look nice in the context of a university student. In its *standard form* (i.e., derived from the `Person` class), it reads as:

```
def greet(self):
    print("Hello!")
    print("My name is", self.first_name, self.last_name)
    print("I am", self.age, "years old")
    print("I live in", self.address)
```

Let's suppose that our `greet()` does not look nice in the context of a university student. In its *standard form* (i.e., derived from the `Person` class), it reads as:

```
def greet(self):
    print("Hello!")
    print("My name is", self.first_name, self.last_name)
    print("I am", self.age, "years old")
    print("I live in", self.address)
```

Inside the `Student` class, we can re-define `greet()` with something like:

```
def greet(self):
    print("Hello!")
    print("My name is", self.first_name, self.last_name)
    print("I am a student at", self.university)
    print("My average grade is", self.get_average())
```

Let's suppose that our `greet()` does not look nice in the context of a university student. In its *standard form* (i.e., derived from the `Person` class), it reads as:

```
def greet(self):
    print("Hello!")
    print("My name is", self.first_name, self.last_name)
    print("I am", self.age, "years old")
    print("I live in", self.address)
```

Inside the `Student` class, we can re-define `greet()` with something like:

```
def greet(self):
    print("Hello!")
    print("My name is", self.first_name, self.last_name)
    print("I am a student at", self.university)
    print("My average grade is", self.get_average())
```

NOTE: This is a particular type of polymorphism known as *method overriding*.

The `greet()` inside `Student` overwrites the inherited one from `Person`. Indeed, we now have:

```
# Assuming the Student class is updated with the new greet method
student1 = Student("Francesco", "Totti", "LUISS", "123456", 30, "Viale Romania")
student1.add_exam(30)
student1.add_exam(25)
student1.add_exam(18)
student1.greet()
```

```
Hello!
My name is Francesco Totti
I am 30 years old
I live in Viale Romania
```

Due to polymorphism, the Python interpreter automatically recognises that the `greet()` method has been overwritten in the child class. So, it uses the one defined in the child class.

As instead, for objects pertaining to the parent class, the `greet()` method remains untouched.

Due to polymorphism, the Python interpreter automatically recognises that the `greet()` method has been overwritten in the child class. So, it uses the one defined in the child class.

As instead, for objects pertaining to the parent class, the `greet()` method remains untouched.

What does this code do?

```
student1 = Student("Francesco", "Totti", "LUISS", "123456", 30, "Viale Romania")

student1.add_exam(30)   # we have student1.exam_grades = [30]
student1.add_exam(25)   # we have student1.exam_grades = [30, 25]
student1.add_exam(18)   # we have student1.exam_grades = [30, 25, 18]

person1 = Person("Daniele", "De Rossi")
list_of_persons = [student1, person1]
for item in list_of_persons:
    item.greet()
```

```
Hello!
My name is Francesco Totti
I am 30 years old
I live in Viale Romania
Hello!
My name is Daniele De Rossi
I am None years old
I live in None
```

Overriding built-in methods

Python classes can override built-in methods to customize behavior.

Common methods to override include:

- `__init__`: Constructor
- `__str__`: String representation (for humans)
- `__repr__`: String representation (for developers)
- `__eq__`: Equality comparison (==)
- `__lt__`, `__gt__`: Less than/greater than comparisons
- `__add__`, `__sub__`: Arithmetic operations

```python
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Point({self.x}, {self.y})"

    def __eq__(self, other):
        if not isinstance(other, Point):
            return False
        return self.x == other.x and self.y == other.y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)
```

```
p1 = Point(1, 2)
p2 = Point(3, 4)
print(p1)                    # Point(1, 2)
print(p1 == Point(1, 2))     # True
print(p1 + p2)               # Point(4, 6)
```

```
Point(1, 2)
True
Point(4, 6)
```